



Um Estudo de Caso do Método Guloso com o Algoritmo De Compressão de Dados de Huffman



Universidade Estadual de Santa Cruz



Departamento de Engenharias e Computação
Área de Computação
Prof. Dr. Paulo André S. Giacomini



Códigos de Huffman

Conceitualização do problema

Um arquivo de dados pode ser representado de diferentes formas, com diferente quantidade de dados.

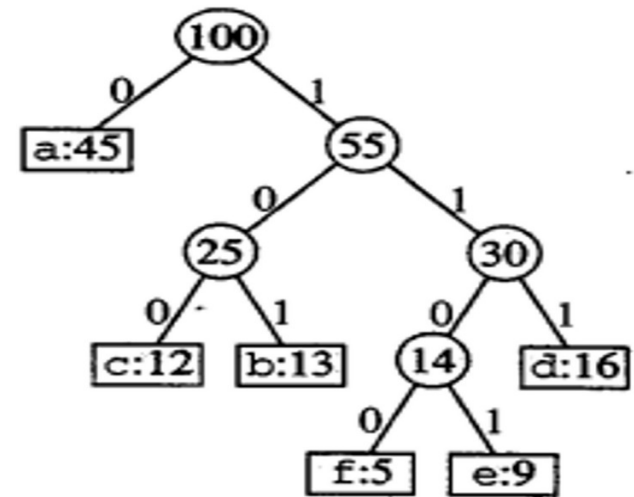
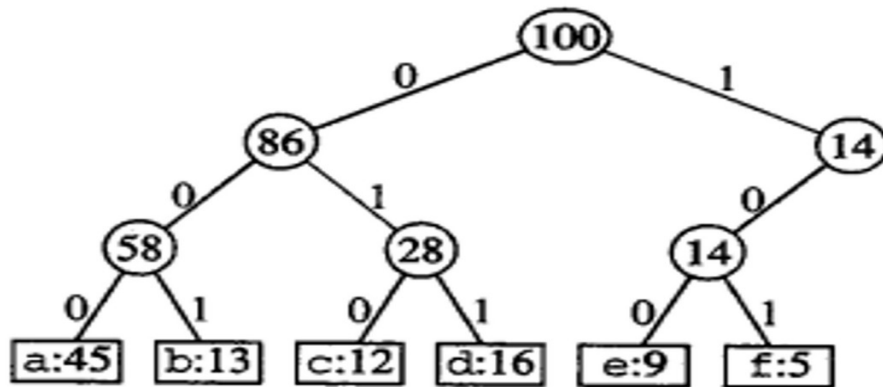
	a	b	c	d	e	f
Frequência (em milhares)	45	13	12	16	9	5
Palavra de código de comprimento fixo	000	001	010	011	100	101
Palavra de código de comprimento variável	0	101	100	111	1101	1100

Fonte: Cormen et al. (2002)



Códigos de Huffman

A árvore ótima é completa. Além disso, cada árvore pode ser percorrida para determinar o código de cada caractere antes e depois do processo de compressão.



Fontes: Cormen et al. (2002)



Códigos de Huffman

- Taxa de compressão
 - $f(c)$ é a frequência do caractere
 - $d_T(c)$ é a distância da raiz até a folha do caractere
 - TC é a taxa de compressão
 - $B_i(T)$ é a quantidade inicial de bits
 - $B_f(T)$ é a quantidade final de bits

$$TC = B_f(T) / B_i(T) * 100\%$$

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

Fonte: Cormen et al. (2002)



Códigos de Huffman

- Taxa de compressão
 - Exemplo

$$B(T)_i = 3 * (45 + 13 + 12 + 16 + 9 + 5) = 300$$

$$B(T)_f = 45 * 1 + 3 * (12 + 13 + 16) + 4 * (5 + 9) = 224$$

$$TC = (300 - 224) / 300 * 100\% = 25,33\%$$



Códigos de Huffman

HUFFMAN(*C*)

1 $n \leftarrow |C|$

2 $Q \leftarrow C$

3 for $i \leftarrow 1$ **to** $n - 1$

4 **do** alocar um novo nó z

5 $esquerda[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$

6 $direita[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$

7 $f[z] \leftarrow f[x] + f[y]$

8 $\text{INSERT}(Q, z)$

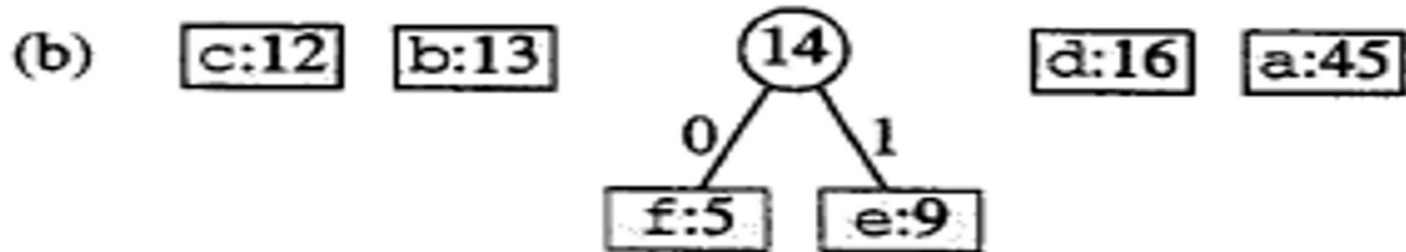
9 return $\text{EXTRACT-MIN}(Q)$

▷ Retornar a raiz da árvore.



Códigos de Huffman

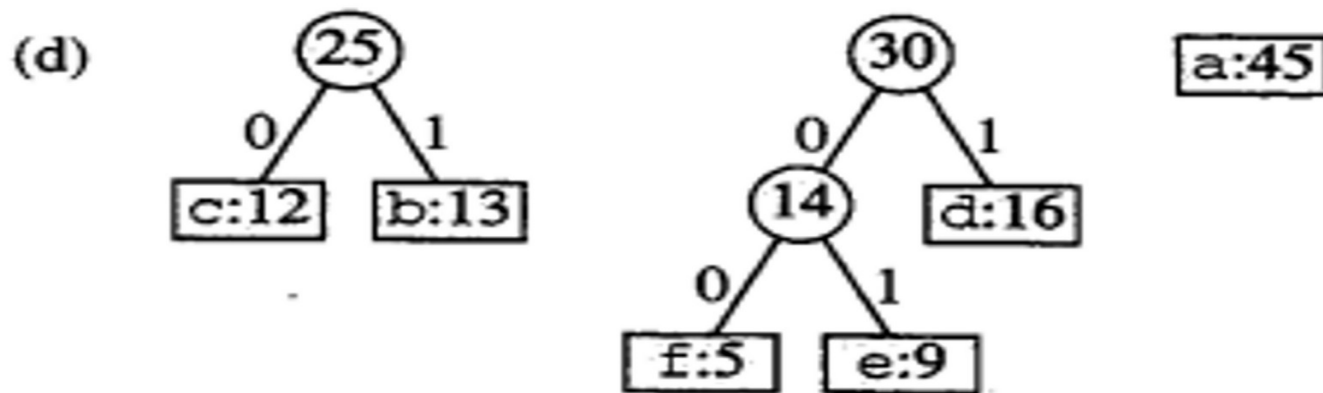
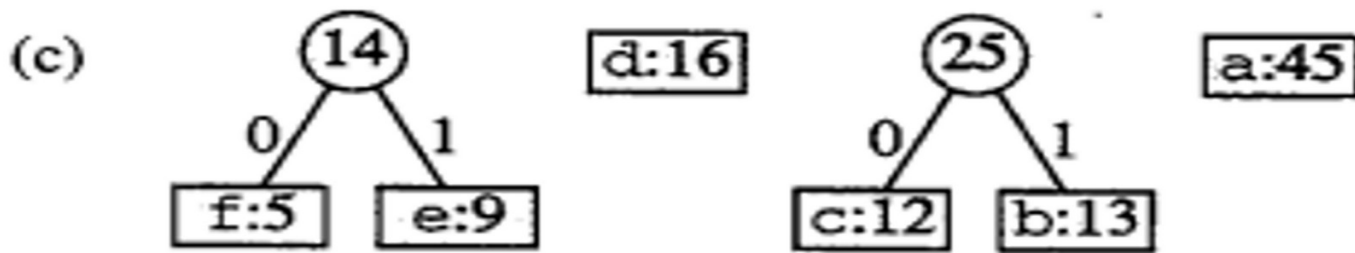
(a) f:5 e:9 c:12 b:13 d:16 a:45



Fonte: Cormen et al. (2002)



Códigos de Huffman

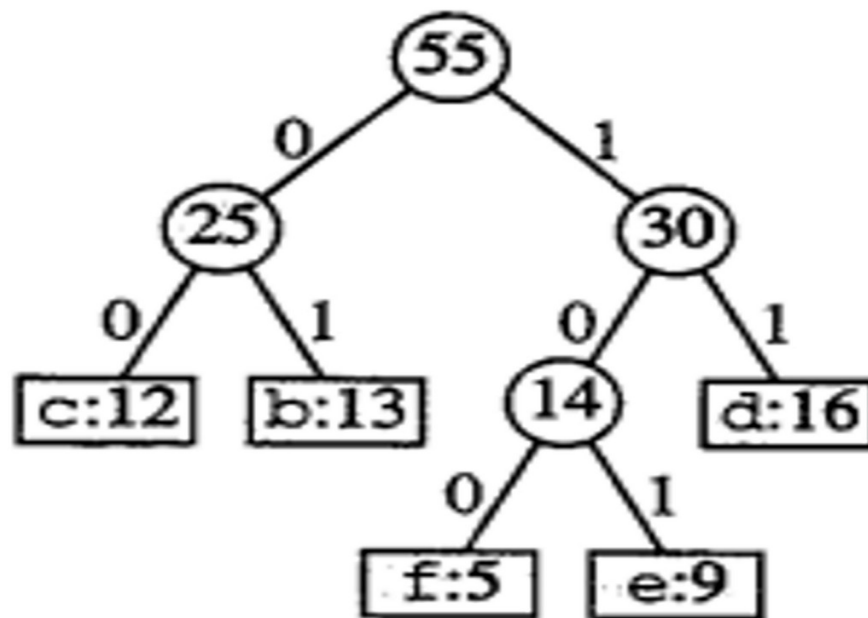




Códigos de Huffman

(e)

a:45



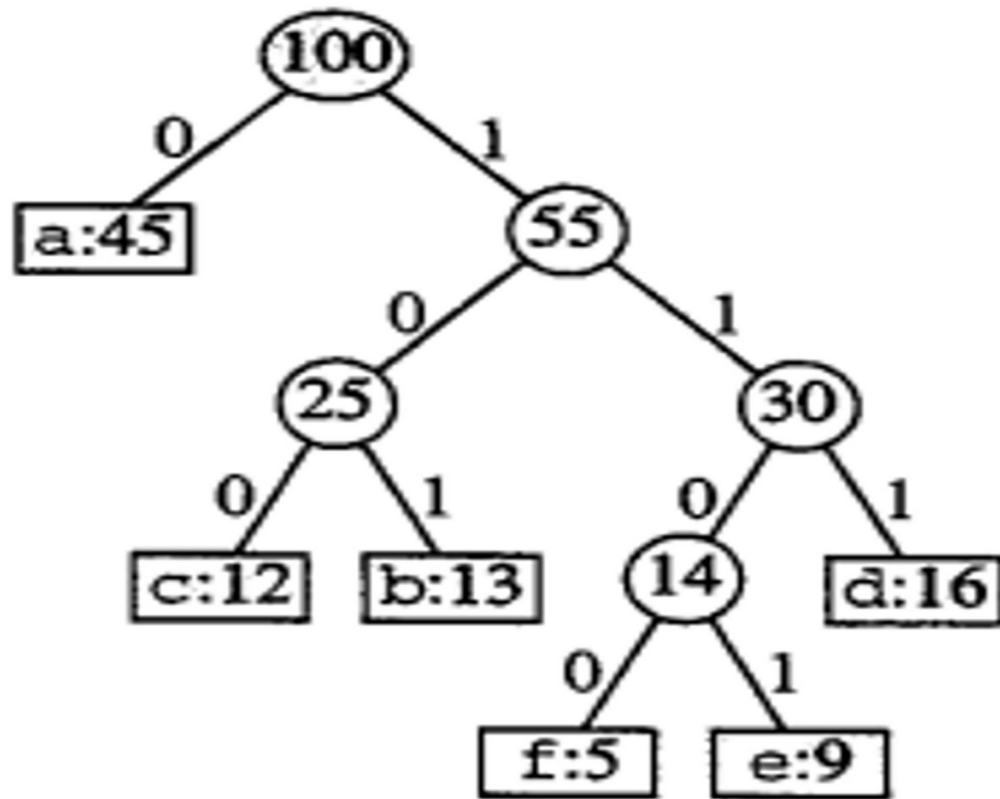
Fonte: Cormen et al. (2002)



Códigos de Huffman

Para encontrar o código de cada caractere após a compressão, basta percorrer a árvore gerada pelo algoritmo de Huffman da raiz até o respectivo caractere, e concatenar os zeros e uns encontrados à direita do novo código em construção.

(f)





EXERCÍCIO

1 – Sejam os caracteres a seguir, e suas respectivas quantidades, em um arquivo de dados. Determine:

- a) A quantidade inicial de bits necessária para representar cada caractere antes da compressão;
- b) A árvore inicial;
- c) A obtenção passo a passo da árvore final;
- d) $B_i(T)$;
- e) $B_f(T)$;
- f) A taxa de compressão;
- g) O código de cada caractere antes e depois da compressão.

A: O resto da divisão do seu número de matrícula por 72; Se $A=0$, $A \leftarrow 10$;
B: O resto da divisão do seu número de matrícula por 39; Se $B=0$, $B \leftarrow 20$;
C: $B+5$;
D: $A+8$;
E: $D*2$;
F: $E-2$;
G: $F*3$;

a A b B c C d D e E f F g G



BIBLIOGRAFIA

CORMEN, T. H.; Desmistificando algoritmos. Elsevier. 2014.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.; Introduction to Algorithms. MIT Press. Third Edition. 2009.

DROZDEK, Adam. Estruturas de Dados e Algoritmos em C++. Thomson Pioneira, 2001.

KNUTH, Donald E.; The Art of Computer Programming – Sorting and Searching. 2o Ed. Boston: Addison-Wesley. 1998.

LEISERSON, Charles E.; STEIN, Clifford; RIVEST, Ronald L.; CORMEN, Thomas H. Algoritmos - Trad. 2ª Ed. Americana, Editora Campus, 2002.

PREISS, Bruno. Estruturas de Dados e Algoritmos. Editora Campus, 2001.

SKIENA, Steven S. The Algorithm Design Manual. Springer-Verlag, 1997. Online: <http://www2.toki.or.id/book/AlgDesignManual/>

SKIENA, Steven S.; REVILLA, Miguel A. Programming Challenges. Springer, 2003.

SKIENA, Steven S.; REVILLA, Miguel A. Programming Challenges. Springer. 2003 Online: http://acm.cs.buap.mx/downloads/Programming_Challenges.pdf

SKIENA, Steven S. The Algorithm Design Manual. Springer, 2008. Online: <https://www.inf.ufpr.br/andre/textos-CI1355-CI355/TheAlgorithmDesignManual.pdf>